TERRAFORM TFVARS AND STATE FILE MODIFICATIONS

TFVARS

1. terraform.tfvars is a file used to store values for Terraform variables.
2. Instead of hardcoding values inside your Terraform code, you define variables and put their actual values in the .tfvars file.
3. terraform.tfvars is a file used to provide values to Terraform input variables. It helps separate configuration values from code, makes infrastructure reusable across environments like dev, test, and prod, and avoids entering variable values manually during every Terraform execution.

Why do we use tfvars

1. Separate code from values
2. Reuse the same code for different environments like dev/prod
3. Makes code cleaner

MODULE FILES

TERRAFORM-B30 (project-folder)

```
├── main.tf
├── output.tf
├── var.tf
├── terraform.tfvars
│
└── modules/
  ├── vpc/
  │   ├── main.tf
  │   ├── variables.tf
  │   └── outputs.tf
  │
  ├── ec2/
  │   ├── main.tf
  │   ├── variables.tf
  │   ├── outputs.tf
  │   └── --user_data.sh
```

VPC

1.main.tf

```
resource "aws_vpc" "my_vpc" {
   cidr_block = var.vpc_cidr
   tags = {
      Name = "my-vpc"
   }
}

resource "aws_subnet" "public_subnet" {
   vpc_id = aws_vpc.my_vpc.id
   cidr_block = var.public_subnet_cidr
   availability_zone = var.public_az
   map_public_ip_on_launch = true
   tags = {
      Name = "public-subnet"
   }
}

resource "aws_subnet" "private_subnet" {
   vpc_id = aws_vpc.my_vpc.id
   cidr_block = var.private_subnet_cidr
   availability_zone = var.private_az
   tags = {
      Name = "private-subnet"
   }
}

resource "aws_internet_gateway" "IGW" {
   vpc_id = aws_vpc.my_vpc.id
   tags = {
      Name = "IGW"
   }
}

resource "aws_eip" "nat_eip" {
   domain = "vpc"
   tags = {
      Name = "nat-eip"
```

```
  }
}

resource "aws_nat_gateway" "nat_gateway" {
  subnet_id = aws_subnet.public_subnet.id
  allocation_id = aws_eip.nat_eip.id
  tags = {
    Name = "nat-gateway"
  }
}

resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.my_vpc.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.IGW.id
  }
  tags = {
    Name = "public-rt"
  }
}

resource "aws_route_table_association" "public_rt_assoc" {
  subnet_id = aws_subnet.public_subnet.id
  route_table_id = aws_route_table.public_rt.id
}

resource "aws_route_table" "private_rt" {
  vpc_id = aws_vpc.my_vpc.id
  route {
    cidr_block = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.nat_gateway.id
  }
  tags = {
    Name = "private-rt"
  }
}

resource "aws_route_table_association" "private_rt_assoc" {
  subnet_id = aws_subnet.private_subnet.id
```

```
  route_table_id = aws_route_table.private_rt.id
}

resource "aws_security_group" "sg" {
  name = var.sg_name
  description = "my-security-group"
  vpc_id = aws_vpc.my_vpc.id

  ingress {
    from_port = var.ingress_http
    to_port = var.ingress_http
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port = var.ingress_ssh
    to_port = var.ingress_ssh
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port = 0
    protocol = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = var.sg_name
  }
}
```

2.variable.tf

```
variable "vpc_cidr" {}
variable "public_subnet_cidr" {}
variable "private_subnet_cidr" {}
variable "public_az" {}
variable "private_az" {}
```

```
variable "sg_name" {}
variable "ingress_http" {}
variable "ingress_ssh" {}
```

3.output.tf

```
output "public_subnet_id" {
  value = aws_subnet.public_subnet.id
}
output "private_subnet_id" {
  value = aws_subnet.private_subnet.id
}
output "vpc_id" {
  value = aws_vpc.my_vpc.id
}
output "sg_id" {
  value = aws_security_group.sg.id
}
```

EC2

1.main.tf

```
resource "aws_instance" "public_instance" {
  ami = var.ami
  instance_type = var.instance_type
  key_name = var.key_name
  subnet_id = var.public_subnet_id
  vpc_security_group_ids = [var.sg_id]
  user_data = file("/root/terraform-b30/module/ec2/user-data.sh")
  associate_public_ip_address = true
  tags = {
    Name = "public-instance"
  }
}

resource "aws_instance" "private_instance" {
  ami = var.ami
  instance_type = var.instance_type
  key_name = var.key_name
  subnet_id = var.private_subnet_id
  vpc_security_group_ids = [var.sg_id]
  user_data = file("/root/terraform-b30/module/ec2/user-data.sh")
  tags = {
```

```
    Name = "private-instance"
  }
}
```

2.variable.tf

```
variable "ami" {}
variable "instance_type" {}
variable "key_name" {}
variable "sg_id" {}
variable "public_subnet_id" {}
variable "private_subnet_id" {}
```

3.output.tf

```
output "public_ip" {
  value = aws_instance.public_instance.public_ip
}
output "public_dns" {
  value = aws_instance.public_instance.public_dns
}
```

4.user-data.sh

```
!#/bin/bash
sudo apt update -y
sudo apt install apache2 -y
sudo systemctl start apache2
sudo systemctl enable apache2
sudo echo "<h1> Welcome to terraform </h1>" | tee /var/www/html/index.html
```

MAIN FILES

1.main.tf

```
module "vpc" {
 source = "./module/vpc"
 vpc_cidr = var.vpc_cidr
 public_subnet_cidr = var.public_subnet_cidr
 private_subnet_cidr = var.private_subnet_cidr
 public_az = var.public_az
 private_az = var.private_az
 sg_name = var.sg_name
 ingress_http = var.ingress_http
 ingress_ssh = var.ingress_ssh
```

```
}

module "ec2" {
  source = "./module/ec2"
  ami = var.ami
  instance_type = var.instance_type
  key_name = var.key_name
  sg_id = module.vpc.sg_id
  public_subnet_id = module.vpc.public_subnet_id
  private_subnet_id = module.vpc.private_subnet_id
}
```

2.variable.tf

```
variable "vpc_cidr" {}
variable "public_subnet_cidr" {}
variable "private_subnet_cidr" {}
variable "public_az" {}
variable "private_az" {}
variable "sg_name" {}
variable "ingress_http" {}
variable "ingress_ssh" {}

variable "ami" {}
variable "instance_type" {}
variable "key_name" {}
```

3.output.tf

```
output "sg_id" {
   value = module.vpc.sg_id
}
output "public_ip" {
   value = module.ec2.public_ip
}
```

4.provider.tf

```
provider "aws" {
   region = "ap-south-1"
   profile = "config"
}
```

5.terraform.tfvars

```
vpc_cidr = "10.0.0.0/16"
public_subnet_cidr = "10.0.0.0/24"
private_subnet_cidr = "10.0.1.0/24"
public_az = "ap-south-1a"
private_az = "ap-south-1b"
sg_name = "my-security-group"
ingress_http = 80
ingress_ssh = 22

ami = "ami-07a00cf47dbbc844c"
instance_type = "t3.micro"
key_name = "ubuntu"
```

1. git add .
2. git commit -m “modules-tfvars”
3. git push

ON UBUNTU-LOCAL

1. git clone https://github.com/mayurpandit25/terraform-b30.git
2. cd terraform-b30
3. terraform init
4. terraform apply -var-file=’terraform.tfvars’ --auto-approve
5. terraform destroy --auto-approve

Note:
You can also write these in user-data

1. user_data = file("${path.module}/user-data.sh")

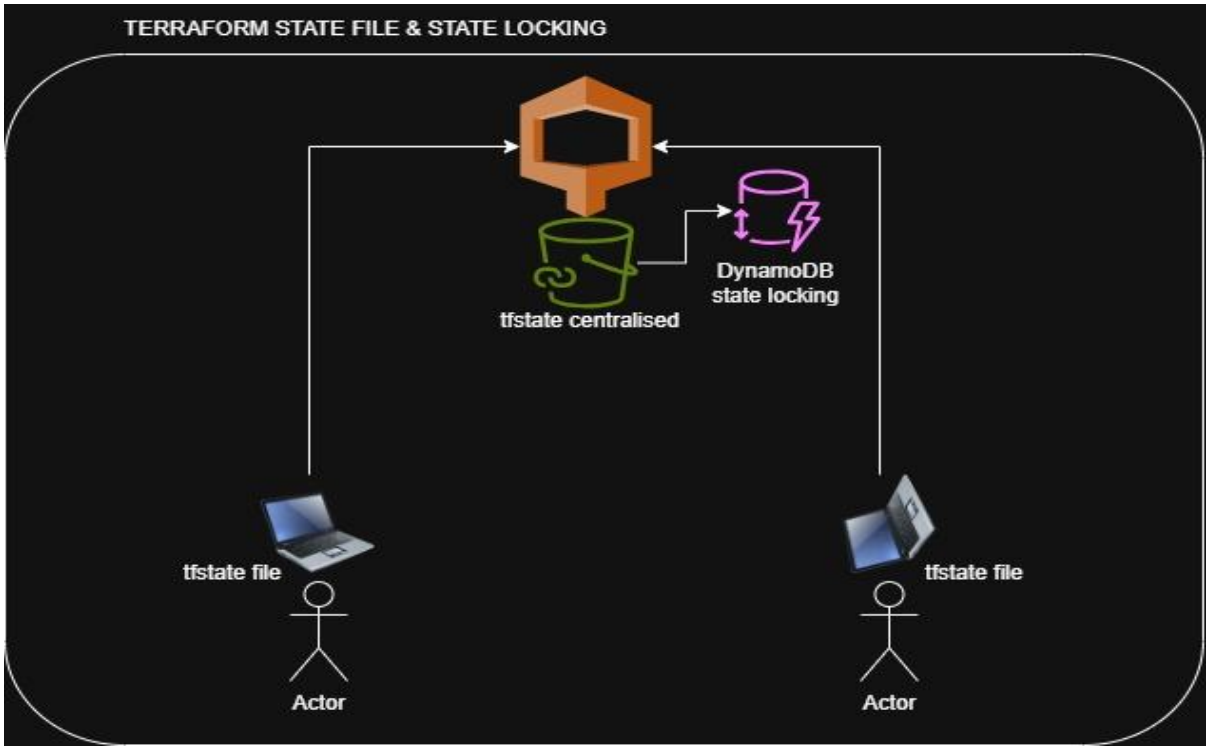
STATE MANAGEMENT (TFSTATE FILE)

TFSTATE FILE

The Terraform state file is a JSON file that stores the current state of infrastructure and helps Terraform track resources, manage dependencies, and determine the changes required during deployments.

TERRAFORM BACKEND

Defines where Terraform state is stored (local, S3, Terraform Cloud, etc.) and how it is managed (locking, encryption, sharing).



PRACTICAL ON TFSTATE & STATE LOCKING

1.create a s3 bucket (amazon-s3-bucket-mayur123)

2.create a DynamoDB table

1. Name: cbz
2. Partition_key: LockID

3.provider.tf

```
provider "aws" {
   region = "ap-south-1"
   profile = "config"
}

terraform {
  backend "s3" {
    bucket = "amazon-s3-bucket-mayur123"
    key    = "terraform.tfstate"
    region = "ap-south-1"
    dynamodb_table = "cbz"
    profile = "config"
```

```
  shared_credentials_files = [ "/root/.aws/credentials" ]
 }
}
```

4.ec2.tf

```
resource "aws_instance" "ec2" {
  ami           = "ami-08d59269edddde222"
  instance_type = "t3.micro"
  key_name      = "ubuntu"
  tags     = {
    Name = "ec2-instance"
  }
}
```

1. git add .
2. git commit -m “backend-s3”
3. git push

ON UBUNTU-LOCAL

1. git clone https://github.com/mayurpandit25/terraform-b30.git
2. cd terraform-b30
3. terraform init
4. terraform apply –auto-approve
5. terraform destroy –auto-approve

TO CHECK THE STATE-LOCKING

1. open the two terminals
2. apply this command on both the terminals at same time
    1. terraform apply --auto-approve
    {you get error: - Error acquiring the state lock}
3. you can check the state lockID in the dynamodb table

Note:

1. Terraform acquires a state lock to protect the state from being written by multiple users at the same time.
2. For most commands, you can disable locking with the "-lock=false" flag, but this is not recommended.

ANOTHER WAY FOR STATE-LOCKING

1. instead of creating dynamodb table we directly use the
   use_lockfile = true
2. so your state-lock file created in the s3 bucket

```
terraform {
  backend "s3" {
    bucket = "amazon-s3-bucket-mayur123"
    region = "ap-south-1"
    key = "terraform.tfstate"
    use_lockfile = true
    profile = "config"
    shared_credentials_files = [ "/root/.aws/credentials" ]
  }
}
```

1. Terraform creates a .tflock file in the same S3 bucket to prevent multiple users from modifying the state file simultaneously.
2. When Terraform starts an operation, it acquires the lock by creating the lock file.
3. After the operation completes, Terraform automatically removes the lock file.
4. This provides state locking without requiring a separate DynamoDB table, simplifying backend configuration.

TERRAFORM LIFECYCLE BLOCK

1. The lifecycle block in Terraform controls how resources are created, updated, or destroyed.
2. It can prevent accidental deletion, ensure zero downtime by creating before destroying, or ignore specific changes so Terraform doesn't try to fix them.

TYPES OF TERRAFORM LIFECYCLE BLOCK

1. create_before_destroy
2. prevent_destroy
3. ignore_changes

1.create_before_destroy

1. Terraform will first create the new resource, then destroy the old one.
2. Used to avoid downtime.
3. Useful when changing something that causes resource replacement.

1.main.tf

```
resource "aws_instance" "web" {
  ami                = "ami-08d59269edddde222"
  instance_type = "t3.micro"

  lifecycle {
    create_before_destroy = true
  }
}
```

1. terraform init
2. terraform apply –auto-approve
3. terraform destroy –auto-approve

2.prevent_destroy

1. Stops Terraform from destroying the resource even if configuration says to remove it.
2. Use when resource is critical.(ex: production DB, IAM roles, S3 buckets)
3. Terraform will throw an error if destroy is attempted.

1.main.tf

```
resource "aws_s3_bucket" "my_bucket" {
  bucket = "important-prod-bucket"

  lifecycle {
    prevent_destroy = true
  }
}
```

1. terraform init
2. terraform apply –auto-approve
3. terraform destroy –auto-approve
   {You get the error bucket not deleted}

3.ignore_changes

1. Tells Terraform to ignore changes in some attributes.
2. Used when values change automatically or outside Terraform.
3. Terraform will not try to fix / update these fields even if they drift.

1.main.tf

```
resource "aws_instance" "ec2" {
  ami          = "ami-05d2d839d4f73aafb"
```

```
  instance_type = "t3.micro"
  tags = {
    Name = "server1"
  }
}
```

2.terraform apply

3.change the tags

```
  tags = {
  Name = "server2"
}
```

4.Now use ignore changes

```
  resource "aws_instance" "ec2" {
  ami                  = "ami-05d2d839d4f73aafb"
  instance_type = "t3.micro"
  key_name        = "ubuntu"
  lifecycle {
    ignore_changes = [
      tags
    ]
  }

  tags = {
    Name = "server2"
  }
}
```

5.terraform plan

{ output:- no changes }

Note:

1.even if u changes

```
  Name = "server2"
```

2.terraform says

I will IGNORE this change
